

成绩	
评阅人	
日期	2024 年 月 号

重庆邮电大学

学生实验实习报告册

学年学期： 2023-2024 学年 ☐ 春 ☒ 秋 学期

课程名称： 人工智能与大数据课程设计

学生学院： 通信与信息工程学院

专业班级： 01012205

学生学号： 2022210410

学生姓名： 雷宇航

联系电话： 18580086873

重庆邮电大学教务处制

课程名称	人工智能与大数据课程设计	课程编号	A2012050
实验地点	YF317	实验时间	2024.12.27
指导教师	胡林		
实验名称	基于神经网络的 MNIST 手写体识别		
评阅人签字		成绩	
一、实验目的 1、学习导入和使用 Tensorflow。 2、理解学习神经网络的基本原理。 3、学习使用 Python 实现 MNIST 手写体识别。 二、实验原理 1、掌握神经网络的学习训练方法，并实现 MNIST 手写数字识别。 2、利用训练的神经网络模型预测 MNIST 测试集，可视化标签和模型输出结果。 三、使用软件平台 1、Windows 11 电脑一台。 2、Python、Anaconda、Jupyter Notebook 平台。 四、实验步骤、代码、结果及结果分析（复制代码，结果截图） 1. 实例 1：构建、训练、保存神经网络模型，并输出训练及测试准确率 代码： <pre> 1. from tensorflow.examples.tutorials.mnist import input_data 2. import tensorflow.compat.v1 as tf 3. tf.disable_v2_behavior() 4. from typing import Any,Callable,Dict, Mapping, Optional, Sequence, Tuple 5. from typing_extensions import OrderedDict 6. import warnings 7. warnings.filterwarnings('ignore') 8. mnist=input_data.read_data_sets("C:\\Users\\86185\\Desktop\\专业课\\大三上\\人工智能大数据\\实验\\实验八\\MNIST_data",one_hot=True) 9. in_x = tf.placeholder(tf.float32, [None, 784], name='x-input') 10.in_y = tf.placeholder(tf.float32, [None, 10], name='y-input') </pre>			

```
11. def init_weights(shape):
12.     return tf.Variable(tf.random_normal(shape, stddev=0.01))
13. h1 = init_weights([784, 16])
14. h2 = init_weights([16, 16])
15. out = init_weights([16, 10])
16. def model(X, h1, h2, out):
17.     h = tf.nn.relu(tf.matmul(X, h1))
18.     h2 = tf.nn.relu(tf.matmul(h, h2))
19.     return tf.matmul(h2, out)
20. mod = model(in_x, h1, h2, out)
21. cost = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(logits=
    mod, labels=in_y))
22. train_op = tf.train.RMSPropOptimizer(0.001, 0.9).minimize(cost)
23. predict_acc = tf.reduce_mean(tf.cast(tf.equal(tf.argmax(mod, 1), tf.a
    rgmax(in_y, 1)), tf.float32))
24. train_num = 1000
25. figure_num = 64
26. saver = tf.train.Saver()
27. sess = tf.Session()
28. sess.run(tf.global_variables_initializer())
29. train_num = int(input('请输入需要训练的次数: '))
30. for step in range(1, train_num+1):
31.     batch_x, batch_y = mnist.train.next_batch(figure_num)
32.     sess.run(train_op, feed_dict={in_x: batch_x, in_y: batch_y})
33.     if step % 1000 == 0:
34.         loss, acc = sess.run([cost, predict_acc], feed_dict={in_x: ba
            tch_x, in_y: batch_y})
35.         print("第%d 次训练"%step, "损失值: {:.6f}".format(loss), "训练
            准确率: {:.3f}%".format(acc*100))
36.         saver.save(sess, './mnist_models/model.ckpt', global_step=step)
37. print("测试准确
    率: {:.3f}%".format(sess.run(predict_acc, feed_dict={in_x: mnist.test
        .images, in_y: mnist.test.labels})*100))
38.
39. import matplotlib.pyplot as plt
40. import numpy as np
41. %matplotlib inline
42. plt.rc("font", family="SimHei", size="12")
43. def display_compare(num):
44.     x_train = mnist.test.images[num, :].reshape(1, 784)
45.     y_train = mnist.test.labels[num, :]
46.     np.set_printoptions(precision=1)
47.     np.set_printoptions(suppress=True)
48.     label = y_train.argmax()
49.     prediction = sess.run(mod, feed_dict={in_x: x_train}).argmax()
50.     if(prediction == label):
```

```

51.     plt.title('预测值: %d, 标签: %s, 预测正确!' %(prediction, label))
52.     else:
53.         plt.title('预测值: %d, 标签: %s, 预测错误!' %(prediction, label))
54.     plt.imshow(x_train.reshape([28, 28]), cmap=plt.get_cmap('gray_r'
    ))
55.     plt.savefig(f"test08_{num}.png")
56.     plt.show()
57.     module_file = tf.train.latest_checkpoint('./mnist_models/')
58.     saver.restore(sess, module_file)
59. while True:
60.     n =int(input('请输入想要查看的图像编号: '))
61.     display_compare(n)
62.     print('输入任意数字继续, 如果要退出查看, 请直接回车。')
63.     flag =bool(input())
64.     if flag==False:
65.         break

```

输出:

```

请输入需要训练的次数: 2000
第 1000次训练 损失值: 0.902179 训练准确率:75.000%
第 2000次训练 损失值: 0.370036 训练准确率:93.750%
测试准确率:86.450%

```

图 1 手写体识别神经网络训练 2000 次准确率 and 测试准确率

```

请输入需要训练的次数: 5000
第 1000次训练 损失值: 1.104776 训练准确率:67.188%
第 2000次训练 损失值: 0.352333 训练准确率:87.500%
第 3000次训练 损失值: 0.254293 训练准确率:95.312%
第 4000次训练 损失值: 0.314322 训练准确率:90.625%
第 5000次训练 损失值: 0.139557 训练准确率:96.875%
测试准确率:92.220%

```

图 2 手写体识别神经网络训练 5000 次准确率 and 测试准确率

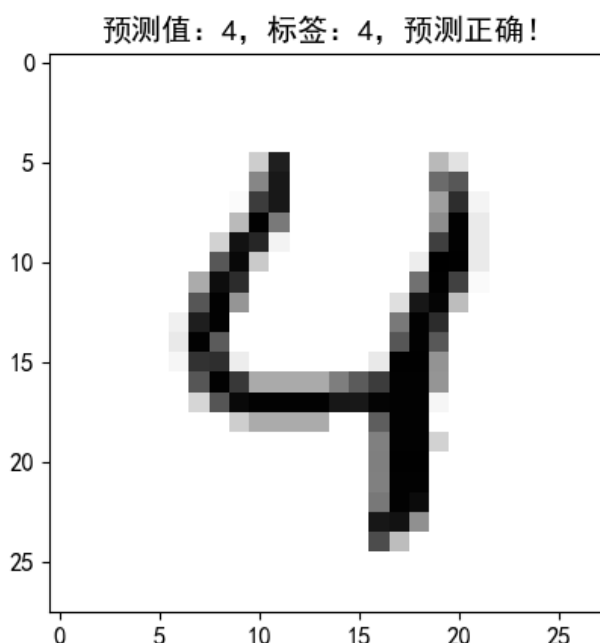


图 3 编号 04 手写体识别

预测值：4，标签：5，预测错误！

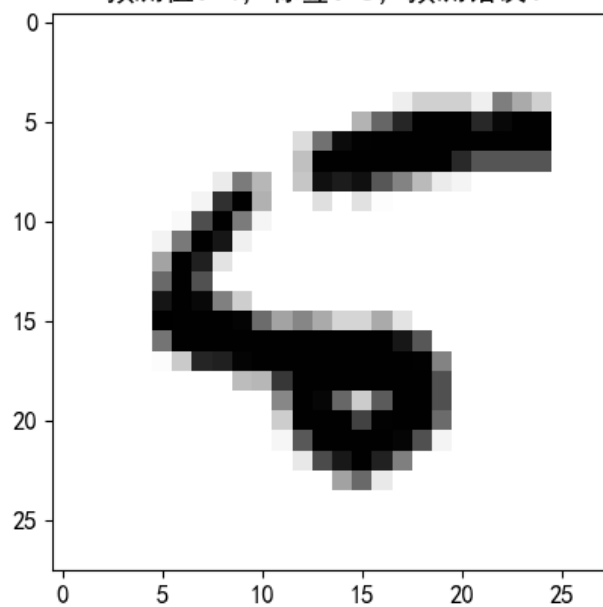


图 4 编号 08 手写体识别

预测值：6，标签：6，预测正确！

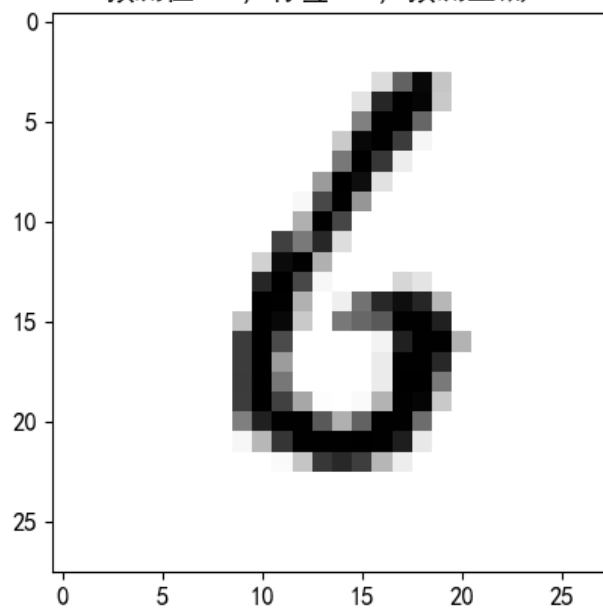


图 5 编号 50 手写体识别

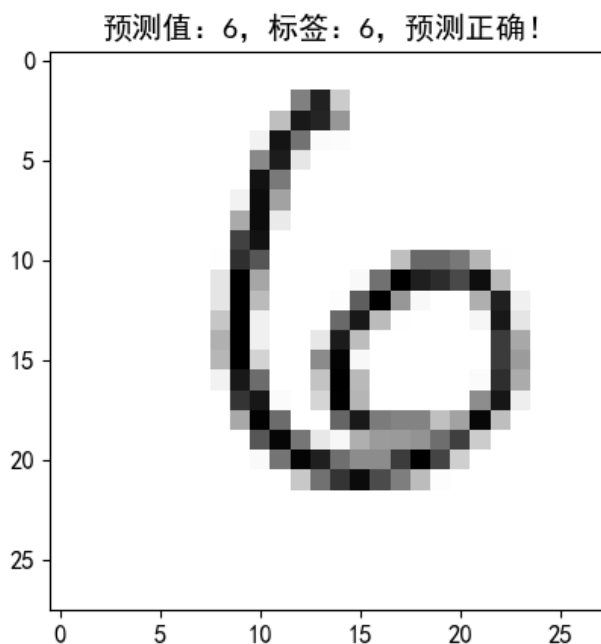


图 6 编号 100 手写体识别

五、碰到的问题及解决办法

在实例 1 中, 导入 `import tensorflow as tf` 时, 会出现 `cannot import name 'Callable' from 'typing_extensions'`。通过找到 `function_type.py` 文件, 将

```
1. from typing import Any,Callable,Dict, Mapping, Optional, Sequence, Tuple
```

修改为

```
1. from typing_extensions import Any,Callable,Dict, Mapping, Optional, Sequence, Tuple
```

并在程序中将 `typing` 和 `typing_extensions` 导入

```
1. from typing import Any,Callable,Dict, Mapping, Optional, Sequence, Tuple
2. from typing_extensions import OrderedDict
```

在安装 `tensorflow` 时出现了 `No module named 'tensorflow.examples'` 的问题。通过在 `tensorflow` 的安装路径下安装 `examples`, 可以解决这个问题。

六、实验心得体会

本次实验极大地增强了我对 `Python` 编程的理解, 特别是在通过实践掌握了 `TensorFlow` 的导入与使用技巧后。我不仅深入理解了神经网络的基本原理, 还成功运用 `Python` 实现了 `MNIST` 手写体识别系统。

在实验过程中, 我首先读取了 `MNIST` 手写数字识别数据集, 它包含 60,000

个训练样本和 10,000 个测试样本。我学习了 TensorFlow 中的变量占位符概念，这些占位符在定义时不赋值，而是在执行运算时传入数据。其中，`dtype` 指定数据类型，`shape` 定义数据形状，`name` 则赋予占位符一个名称。特别地，`None` 在此代表行数（即每个批次送入的图片数量可灵活设置），而 784 则代表列数（因为 MNIST 数据集中的图片分辨率为 28x28 像素，总共 784 个像素点）。

接着，我利用服从正态分布的随机值来初始化参数矩阵，并生成正态分布的神经网络层，从而构建出神经网络训练模型。在模型训练完成后，我计算了预测的准确率。具体做法是，首先使用 `tf.argmax(mod, 1)` 获取模型输出 `mod` 中每一行概率最大的数字，再用 `tf.argmax(in_y, 1)` 得到实际标签对应的数字。随后，通过比较这两个数字是否相等，计算每个批次中的准确值，并最终求得所有批次的平均准确率。

这次实验不仅让我掌握了实用的编程技能，还加深了我对神经网络和机器学习原理的认识。